



Darshan UNIVERSITY

Python Programming - 2301CS404

Lab - 15 (2)

Roll No.:310

Name:Ronit Bhadania

Continued..

10) Calculate area of a ractangle using object as an argument to a method.

```
In [1]: class Rectangle:

    def __init__(self, length, width):
        self.length = length
        self.width = width

    def calculate_area(self):
        return self.length * self.width

r1 = Rectangle(10, 5)

area = r1.calculate_area()

print(f"The area of the rectangle is: {area}")
```

The area of the rectangle is: 50

11) Calculate the area of a square.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

```
In [3]: class Square:

    def __init__(self, side_length):
        self.side_length = side_length
```

```

        self.result = 0

    def area(self):
        self.result = self.side_length ** 2
        self.output()

    def output(self):
        print(f"The area of the square with side length {self.side_length} is: {self.result}")

s1 = Square(5)
s1.area()

```

The area of the square with side length 5 is: 25

12) Calculate the area of a rectangle.

Include a Constructor, a method to calculate area named `area()` and a method named `output()` that prints the output and is invoked by `area()`.

Also define a class method that compares the two sides of rectangle. An object is instantiated only if the two sides are different; otherwise a message should be displayed : THIS IS SQUARE.

```

In [7]: class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        calculated_area = self.length * self.width
        self.output(calculated_area)

    def output(self, calculated_area):
        print(f"The area of the rectangle is: {calculated_area}")

    @classmethod
    def create_rectangle(cls, length, width):
        if length == width:
            print("THIS IS SQUARE")
            return None
        else:
            return cls(length, width)

print("Attempting to create a 5x5 shape:")
s1 = Rectangle.create_rectangle(5, 5)
if s1:
    s1.area()

print("-" * 20)
print("Attempting to create a 4x6 shape:")
s2 = Rectangle.create_rectangle(4, 6)
if s2:
    s2.area()

```

Attempting to create a 5x5 shape:
THIS IS SQUARE

Attempting to create a 4x6 shape:
The area of the rectangle is: 24

13) Define a class Square having a private attribute "side".

Implement `get_side` and `set_side` methods to access the private attribute from outside of the class.

```
In [9]: class Square:
        def __init__(self, side):
            self.__side = side

        def get_side(self):
            return self.__side

        def set_side(self, new_side):
            if new_side >= 0:
                self.__side = new_side
            else:
                print("Error: Side length cannot be negative.")

s1 = Square(5)
print(f"Initial side: {s1.get_side()}")

s1.set_side(10)
print(f"Updated side: {s1.get_side()}")
```

Initial side: 5
Updated side: 10

14) Create a class Profit that has a method named `getProfit` that accepts profit from the user.

Create a class Loss that has a method named `getLoss` that accepts loss from the user.

Create a class BalanceSheet that inherits from both classes Profit and Loss and calculates the balance. It has two methods `getBalance()` and `printBalance()`.

```
In [12]: class Profit:
        def getProfit(self):
            self.profit = float(input("Enter Profit: "))

        class Loss:
            def getLoss(self):
                self.loss = float(input("Enter Loss: "))

        class BalanceSheet(Profit, Loss):
            def getBalance(self):
                self.balance = self.profit - self.loss

            def printBalance(self):
```

```

        print(f"Total Balance: {self.balance}")

obj = BalanceSheet()
obj.getProfit()
obj.getLoss()
obj.getBalance()
obj.printBalance()

```

Total Balance: 500.0

15) WAP to demonstrate all types of inheritance.

```

In [13]: class A:
        def showA(self):
            print("Class A")

        class B(A):
            def showB(self):
                print("Class B (inherits A)")

        class C:
            def showC(self):
                print("Class C")

        class D(A, C):
            def showD(self):
                print("Class D (inherits A and C)")

        class E(B):
            def showE(self):
                print("Class E (inherits B -> A)")

        class F(A):
            def showF(self):
                print("Class F (inherits A)")

        class G(A):
            def showG(self):
                print("Class G (inherits A)")

        class H(D, E):
            def showH(self):
                print("Class H (Hybrid Inheritance)")

print("Single Inheritance:")
b = B()
b.showA()
b.showB()

print("\nMultiple Inheritance:")
d = D()
d.showA()
d.showC()
d.showD()

print("\nMultilevel Inheritance:")
e = E()
e.showA()
e.showB()

```

```

e.showE()

print("\nHierarchical Inheritance:")
f = F()
g = G()
f.showA()
f.showF()
g.showA()
g.showG()

print("\nHybrid Inheritance:")
h = H()
h.showA()
h.showC()
h.showB()
h.showH()

```

Single Inheritance:

Class A

Class B (inherits A)

Multiple Inheritance:

Class A

Class C

Class D (inherits A and C)

Multilevel Inheritance:

Class A

Class B (inherits A)

Class E (inherits B -> A)

Hierarchical Inheritance:

Class A

Class F (inherits A)

Class A

Class G (inherits A)

Hybrid Inheritance:

Class A

Class C

Class B (inherits A)

Class H (Hybrid Inheritance)

16) Create a Person class with a constructor that takes two arguments name and age.

Create a child class Employee that inherits from Person and adds a new attribute salary.

Override the `init` method in Employee to call the parent class's `init` method using the `super()` and then initialize the salary attribute.

```

In [15]: class Person:
          def __init__(self, name, age):
              self.name = name
              self.age = age

```

```
class Employee(Person):
    def __init__(self, name, age, salary):
        super().__init__(name, age)
        self.salary = salary

e1 = Employee("Ronit", 19, 50000)
print("Name:", e1.name)
print("Age:", e1.age)
print("Salary:", e1.salary)
```

Name: Ronit
Age: 19
Salary: 50000

17) Create a Shape class with a draw method that is not implemented.

Create three child classes Rectangle, Circle, and Triangle that implement the draw method with their respective drawing behaviors.

Create a list of Shape objects that includes one instance of each child class, and then iterate through the list and call the draw method on each object.

```
In [16]: class Shape:
        def draw(self):
            pass

        class Rectangle(Shape):
            def draw(self):
                print("Drawing Rectangle")

        class Circle(Shape):
            def draw(self):
                print("Drawing Circle")

        class Triangle(Shape):
            def draw(self):
                print("Drawing Triangle")

s1 = [Rectangle(), Circle(), Triangle()]
for s in s1:
    s.draw()
```

Drawing Rectangle
Drawing Circle
Drawing Triangle